

HAdaptiveIntegration.jl: Adaptive numerical integration over simplices and orthotopes

Luiz Faria ¹ and Zoïs Moitier ²✉

¹ POEMS, CNRS, Inria, ENSTA, Institut Polytechnique de Paris, Palaiseau, France ² Inria, Unité de Mathématiques Appliquées, ENSTA, Institut Polytechnique de Paris, Palaiseau, France ✉ Corresponding author

DOI: [10.21105/joss.10833](https://doi.org/10.21105/joss.10833)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Gabriele Bozzola](#)  

Reviewers:

- [@jipolanco](#)
- [@xuanzhaogao](#)

Submitted: 27 May 2026

Published: 09 October 2026

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

HAdaptiveIntegration.jl is a Julia ([Bezanson et al., 2017](#)) package that computes the numerical value of an integral over a geometric region in any number of dimensions, automatically refining where the integrand is hard to resolve, and returning both the value and an a posteriori error estimate. More precisely, it performs adaptive numerical integration on simplices (triangles, tetrahedra, and their higher-dimensional analogs) and axis-aligned orthotopes (rectangles, cuboids, and their higher-dimensional analogs), approximating integrals of the form

$$I = \int_{\Omega} f(x) \, dx$$

where $f: \mathbb{R}^d \rightarrow \mathbb{T}$ takes values in a normed real vector space (e.g. $\mathbb{T} = \mathbb{R}, \mathbb{C}, \mathbb{R}^n$), and $\Omega \subset \mathbb{R}^d$ is a simplex or an axis-aligned orthotope.

Its main features are:

- Adaptive integration over simplices and orthotopes of arbitrary dimension;
- Efficient tabulated cubature rules for low-dimensional domains;
- Support for user-defined cubature rules and subdivision strategies; and
- Arbitrary-precision arithmetic (formula-based rules natively; tabulated rules via an optional extension).

Usage

The package is centered on a single function, `integrate(f, domain; kwargs...)`, together with constructors for common domains. The workflow has two steps:

Create a domain using `Simplex` (defined by its vertices) or `Orthotope` (defined by its lower and upper corners), with aliases `Segment`, `Triangle`, `Tetrahedron`, `Rectangle`, `Cuboid`.

```
using HAdaptiveIntegration
triangle = Triangle((0,0), (1,0), (0,1))
rectangle = Rectangle((0,0), (1,1))
```

Integrate using the same interface for both domain types:

```
f(x) = cis(sum(x)) / (sum(abs2, x) + 1e-2)
I, E = integrate(f, triangle)
I, E = integrate(f, rectangle)
```

The function returns a pair `(I, E)`, where `I` is the integral estimate and `E` is an a posteriori error estimate. The main stopping-condition keywords are `atol`, `rtol`, and `maxsubdiv`.

Statement of need

Adaptive numerical integration is a fundamental building block in scientific computing, including finite-element, boundary-element methods, and in parameter studies over integrands with localized features. `HAdaptiveIntegration` provides this for simplices and orthotopes, with a single unified interface, user-extensible cubature, and optional arbitrary-precision arithmetic.

State of the field

`HAdaptiveIntegration` is complementary to the existing Julia ecosystem. `QuadGK.jl` (Johnson, 2013) remains the right choice in one dimension. `HCubature.jl` (Johnson, 2017) is often preferable for medium- to high-dimensional orthotopes, where its estimator-driven axis-aligned bisection can outperform isotropic subdivision if the integrand varies primarily along one direction. `Cuba.jl` (Hahn, 2005) is better suited for high-dimensional problems where stochastic methods dominate. The distinct contribution of `HAdaptiveIntegration` is support for simplices of arbitrary dimension and orthotopes under one API with efficient tabulated rules in low dimensions, a combination not provided by the packages above.

Software design

The package has two submodules: `Domain` (domains and subdivision) and `Rule` (cubature rules, formula-based or tabulated), with `integrate` as the single entry point.

Method

Embedded cubature

At the core of an adaptive numerical integration method is a cubature pair $(\mathcal{H}, \mathcal{L})$, defined on the reference domain $\hat{\omega}$ (the unit simplex or $[0, 1]^d$) by

$$\mathcal{H}(f) = \sum_{1 \leq i \leq H} h_i f(\mathbf{x}_i) \quad \text{and} \quad \mathcal{L}(f) = \sum_{1 \leq i \leq L} \ell_i f(\mathbf{x}_i),$$

where $\mathbf{x}_i$ are the nodes, h_i and ℓ_i the respective weights, and $H > L$. The $\mathcal{L}$ rule reuses the first L nodes of $\mathcal{H}$ (hence *embedded*) with generally different weights $\ell_i \neq h_i$, so the pair costs only H evaluations rather than $H + L$. The two rules have polynomial exactness orders $k_h > k_\ell$ (the highest total degree integrated exactly).

For a domain ω with reference map $\phi: \hat{\omega} \rightarrow \omega$, the *local* estimated integral value I_ω and error E_ω are

$$I_\omega = |\det J_\phi| \mathcal{H}(f \circ \phi) \quad \text{and} \quad E_\omega = |\det J_\phi| \|\mathcal{H}(f \circ \phi) - \mathcal{L}(f \circ \phi)\|,$$

where J_ϕ is the (constant) Jacobian and $\|\cdot\|$ is the norm on $\mathbb{T}$.

Default rules per domain are summarized in Table 1 and visualized in Figure 1.

Table 1: Default embedded cubature rules by domain, where k_h and k_ℓ are the polynomial exactness orders of the high and low rules.

Dimension	Domain	k_h	k_ℓ	Reference
1d	Segment	23	13	(Laurie, 1997)
2d	Triangle	8	5	(Laurie, 1982)
2d	Rectangle	7	5	(Genz & Malik, 1980)
3d	Tetrahedron	7	5	(Grundmann & Möller, 1978)

Dimension	Domain	k_h	k_ℓ	Reference
3d	Cuboid	9	7	(Berntsen & Espelid, 1988)
nd	Simplex	7	5	(Grundmann & Möller, 1978)
nd	Orthotope	7	5	(Genz & Malik, 1980)

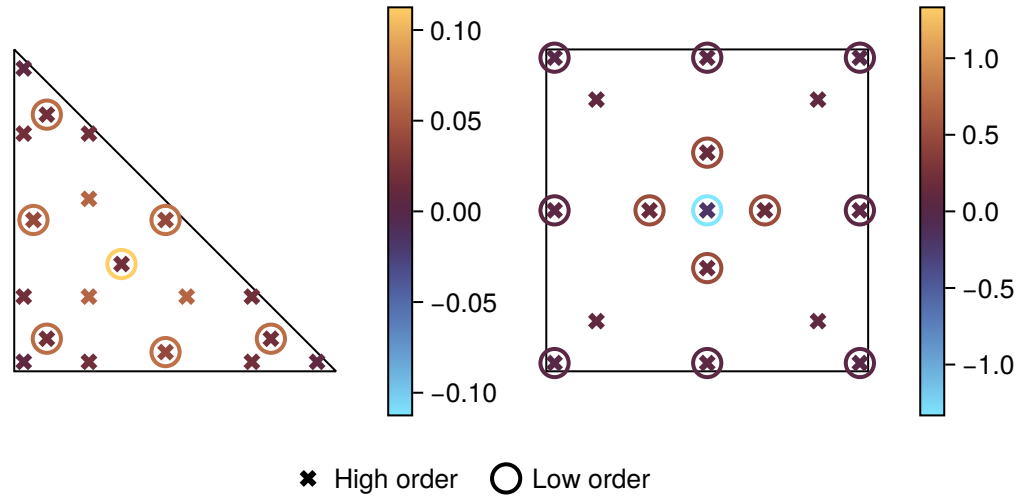


Figure 1: Plots of the default embedded cubature rules used for the triangle and rectangle. Cross markers show the $\mathcal{H}$ nodes and circle markers the $\mathcal{L}$ shared nodes reused by $\mathcal{L}$; color encodes the weight.

The adaptive algorithm

Given a function f and an initial domain Ω , the adaptive algorithm constructs a sequence of nested partitions. It starts with $\mathcal{P}_0 = \{\Omega\}$ and iterates by

$$\mathcal{P}_{n+1} = [\mathcal{P}_n \setminus \{\omega^*\}] \cup \{\omega_1^*, \dots, \omega_{2^d}^*\}, \quad \forall n \in \mathbb{N},$$

where ω^* is chosen such that $E_{\omega^*} = \max\{E_\omega : \omega \in \mathcal{P}_n\}$, and $\omega_1^*, \dots, \omega_{2^d}^*$ are subdomains given by a partition of ω^* . In dimension d , orthotopes are bisected along each axis and simplices by midpoint edge refinement (Gonçalves et al., 2006), both producing 2^d subdomains.

For the sequence $(\mathcal{P}_n)_{n \in \mathbb{N}}$, we define the global integral and error estimators $\mathcal{I}_n$ and $\mathcal{E}_n$ by

$$\mathcal{I}_n = \sum_{\omega \in \mathcal{P}_n} I_\omega \quad \text{and} \quad \mathcal{E}_n = \sum_{\omega \in \mathcal{P}_n} E_\omega.$$

The process stops when $\mathcal{E}_n \leq \text{atol}$ (absolute tolerance) or $\mathcal{E}_n \leq \text{rtol} \|\mathcal{I}_n\|$ (relative tolerance), or $n = \text{maxsubdiv}$ (maximum number of subdivisions), and returns $(\mathcal{I}_n, \mathcal{E}_n)$.

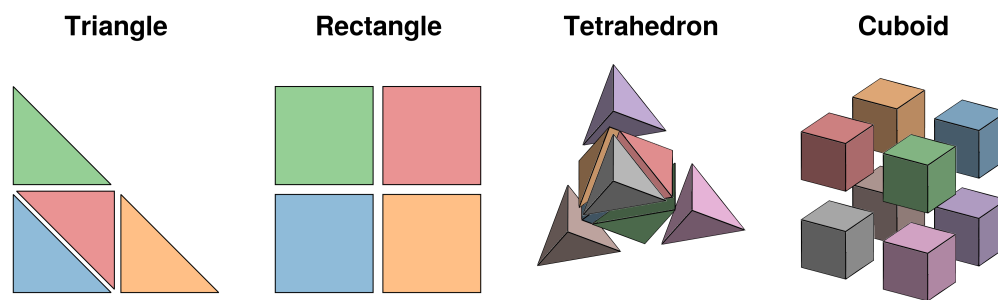


Figure 2: One level of isotropic subdivision for a triangle and tetrahedron split by joining edge midpoints, and a rectangle and cuboid bisected along each axis.

Implementation

The only domain structs are $\text{Simplex}\{D, T, N\}$ (with $N = D + 1$ vertices) and $\text{Orthotope}\{D, T\}$ (low and high corners); `Segment`, `Triangle`, `Tetrahedron`, `Rectangle`, and `Cuboid` are type aliases, so dimension-specific defaults are selected by ordinary dispatch rather than run-time branching. Vertices and cubature nodes are `SVectors`, so the dimension D is part of the type and the inner evaluation loop is stack-allocated and specialized per (D, T) . Default rules are built by `@generated` functions, so the `EmbeddedCubature` for a given element type is constructed once at compile time. An `EmbeddedCubature` stores the shared nodes and separate high- and low-order weight vectors, the first L nodes being those of the low-order rule. Rules are tabulated on the reference domain and mapped by `map_from_reference`, which returns $(\phi, |\det J_\phi|)$ with constant Jacobian.

The adaptive set lives in a max binary heap from [DataStructures.jl](#), ordered by local error, and `integrate` maintains $(\mathcal{I}_n, \mathcal{E}_n)$ incrementally. When computing many integrals of the same type, the heap can be pre-allocated and passed via the `buffer` keyword so that, provided the integrand does not allocate, `integrate` performs no allocations; see the documentation for a zero-allocation benchmark. Defaults are `atol = 0`, `rtol = sqrt(eps(T))` for element type T , and `maxsubdiv = 2^{13+D}`; if `maxsubdiv` is reached, a warning is issued and the current estimate is returned. The same code path runs with `Float64`, `BigFloat`, and [Unitful.jl](#) quantities. Extensibility is provided through the `rule` and `norm` keywords on `integrate`, which let users plug in their own cubature and error norm.

Extended precision

As noted above, `integrate` supports arbitrary precision. Only the rules from Grundmann & Möller (1978) and Genz & Malik (1980) are formula-based and already arbitrary-precision; the others are tabulated at quadruple precision and therefore incompatible with it. `HAdaptiveIntegration` addresses this with the optional `IncreasePrecisionExt` extension, which refines a tabulated rule's nodes and weights by a least-squares Newton iteration on the polynomial-exactness (moment) conditions for both $\mathcal{H}$ and $\mathcal{L}$, with Jacobians from `ForwardDiff.jl` (Revels et al., 2016), following (Xiao & Gimbutas, 2010, sec. 2.3). The full construction, including the definition of the refinement operator and its stopping criteria, is given in the documentation.

Remark. The monomial basis is convenient but poorly conditioned, so achieving precision ε requires `BigFloat` arithmetic at a higher internal precision $\eta < \varepsilon$; an L^2 -orthogonal basis would improve conditioning.

Remark. Symmetry is not enforced in the Newton iteration; a rule refined from precision ε to $\eta < \varepsilon$ therefore retains its symmetry only up to ε .

Research impact statement

The repository includes API documentation, examples (buffer pre-allocation, callback mechanism, custom cubature, arbitrary-precision workflows), an automated test suite, and continuous integration covering tests, documentation, and linting. HAdaptiveIntegration has been featured in the [Julia world newsletter](#), interfaced via [Integrals.jl](#) (SciML ecosystem), used as a backend in Anderson et al. (2024), and adopted as a dependency of [Meshes.jl](#).

Example gallery

We showcase the package on integrands with localized features constructed from the mollified integrand

$$\rho_\delta(\psi(\mathbf{x})) = \frac{1}{\delta^c} \rho\left(\frac{\psi(\mathbf{x})}{\delta}\right), \quad \rho(r) = \frac{1}{\sqrt{2\pi}} e^{-r^2/2},$$

where ψ is a level-set function vanishing on $\Gamma = \psi^{-1}(0)$, c is the codimension of Γ , so that the total mass stays $\mathcal{O}(1)$ as $\delta \rightarrow 0$; $\rho_\delta \circ \psi$ varies rapidly near Γ . We consider three canonical geometries:

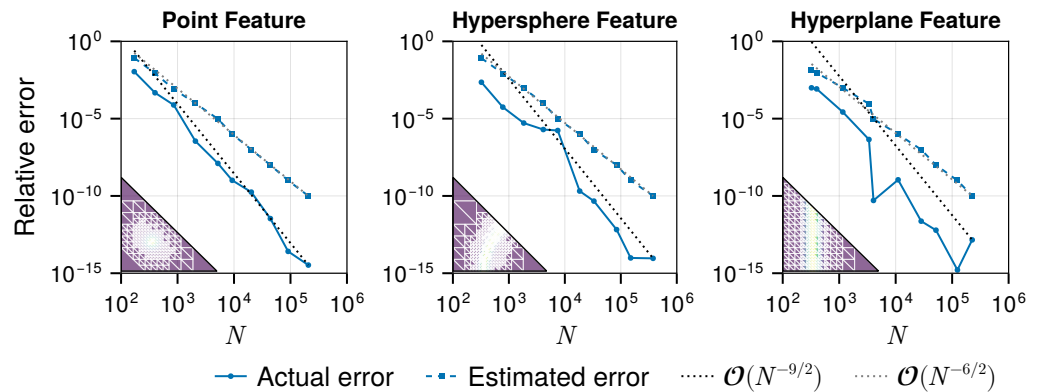
- **Point:** $\psi(\mathbf{x}) = \|\mathbf{x} - \mathbf{z}_0\|_2$, codimension $c = d$ (point refinement);
- **Hypersphere:** $\psi(\mathbf{x}) = \|\mathbf{x} - \mathbf{z}_1\|_2^2 - r^2$, codimension $c = 1$ (curve/surface refinement);
- **Hyperplane:** $\psi(\mathbf{x}) = x_1 - a$, codimension $c = 1$ (axis-aligned hyperplane refinement).

We set $\delta = 0.05$, $\mathbf{z}_0 = \frac{1}{\pi}(1, \dots, 1)$, $\mathbf{z}_1 = (1, 0, \dots, 0)$, $r = 2/3$, $a = 1/\pi$; the axis-aligned hyperplane benchmarks isotropic versus estimator-driven subdivision. Convergence plots sweep $\text{rtol} = 10^{-i}$ ($i = 1, \dots, 10$; up to 8 in 3D), recording the total number of evaluations N , the returned error estimate, and the actual error against a reference solution computed at $\text{rtol} = 10^{-12}$.

When one evaluation of f dominates the per-node overhead of the algorithm (a few clock cycles), the wall-clock time is roughly N times the cost of a single evaluation, so N is a hardware-independent proxy for runtime.

Simplices

Figure 3 shows convergence on the unit triangle ($d = 2$, Radon-Laurie rule (Laurie, 1982)) and tetrahedron ($d = 3$, Grundmann-Möller rule (Grundmann & Möller, 1978), available in arbitrary dimensions). Two observations hold across all features and both geometries: the estimated error reliably tracks the actual error, confirming a sound a posteriori indicator; and once the feature is resolved, errors follow $\mathcal{O}(N^{-(k+1)/d})$ with $k = k_h$ or k_ℓ and the exponent implied by $N \propto h^{-d}$.



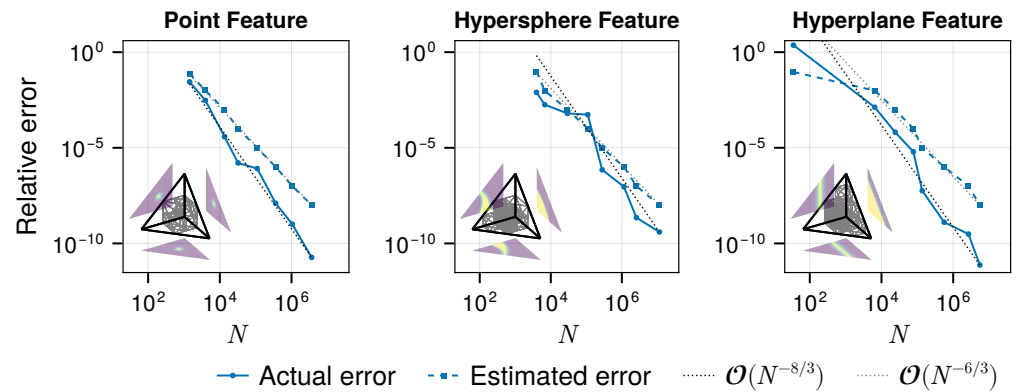
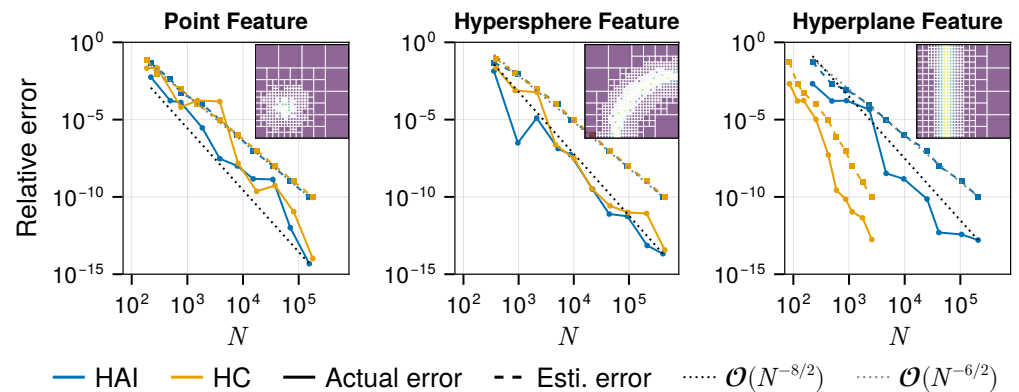


Figure 3: Convergence of the actual and estimated errors for the point, hypersphere, and hyperplane features on the unit triangle (top) and unit tetrahedron (bottom) versus the number of evaluations (N). Insets show the sub-domains and the integrand as max-intensity projections onto the coordinate planes.

Orthotopes and comparison with HCubature.jl

Figure 4 compares HAdaptiveIntegration against HCubature.jl on the unit square and cube. In 2D, both use the Genz-Malik rule (Genz & Malik, 1980), isolating the subdivision strategy; in 3D, the cubature rules also differ (Berntsen-Espelid (Berntsen & Espelid, 1988) for HAdaptiveIntegration). For the point and hypersphere features the solvers are comparable in both dimensions. For the hyperplane, HCubature.jl has a clear advantage because its estimator refines exclusively along x_1 rather than bisecting uniformly in all d directions, and this gap grows with d , motivating anisotropic splitting as future work.



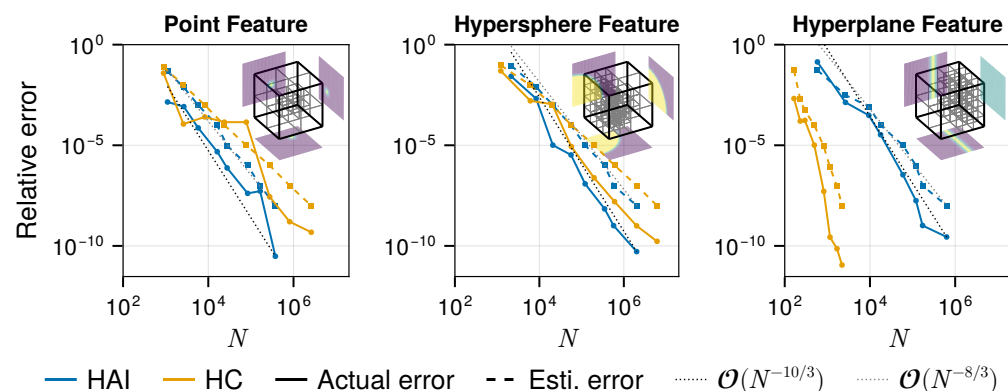


Figure 4: Convergence of the actual and estimated errors for the point, hypersphere, and hyperplane features on the unit square (top) and the unit cube (bottom), comparing HAdaptiveIntegration (HAI) with HCubature.jl. Insets show the sub-domains and the integrand as max-intensity projections onto the coordinate planes.

Acknowledgments

We thank the reviewers and editors for their time and comments, which helped improve both the package and this paper.

AI usage disclosure

Generative AI was used to help develop the code and to draft and edit parts of this manuscript. All text produced with AI assistance was reviewed, revised, and verified by the authors before inclusion in the code or paper.

References

- Anderson, T. G., Faria, L. M., & Pérez-Arancibia, C. (2024). Inti.jl. In *GitHub repository*. GitHub. <https://github.com/IntegralEquations/Inti.jl>
- Berntsen, J., & Espelid, T. O. (1988). On the construction of higher degree three-dimensional embedded integration rules. *SIAM J. Numer. Anal.*, 25(1), 222–234. <https://doi.org/10.1137/0725016>
- Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Rev.*, 59(1), 65–98. <https://doi.org/10.1137/141000671>
- Genz, A. C., & Malik, A. A. (1980). Remarks on algorithm 006: An adaptive algorithm for numerical integration over an N-dimensional rectangular region. *J. Comput. Appl. Math.*, 6(4), 295–302. [https://doi.org/10.1016/0771-050X\(80\)90039-X](https://doi.org/10.1016/0771-050X(80)90039-X)
- Gonçalves, E. N., Palhares, R. M., Takahashi, R. H. C., & Mesquita, R. C. (2006). Algorithm 860: SimpleS – an extension of Freudenthal's simplex subdivision. *ACM Trans. Math. Softw.*, 32(4), 609–621. <https://doi.org/10.1145/1186785.1186792>
- Grundmann, A., & Möller, H. M. (1978). Invariant integration formulas for the n-simplex by combinatorial methods. *SIAM J. Numer. Anal.*, 15(2), 282–290. <https://doi.org/10.1137/0715019>
- Hahn, T. (2005). Cuba—a library for multidimensional numerical integration. *Computer Physics Communications*, 168(2), 78–95. <https://doi.org/10.1016/j.cpc.2005.01.010>

- Johnson, S. G. (2013). QuadGK.jl: Gauss–Kronrod integration in Julia. In *GitHub repository*. GitHub. <https://github.com/JuliaMath/QuadGK.jl>
- Johnson, S. G. (2017). The HCubature.jl package for multi-dimensional adaptive integration in Julia. In *GitHub repository*. GitHub. <https://github.com/JuliaMath/HCubature.jl>
- Laurie, D. P. (1982). CUBTRI: Automatic cubature over a triangle. *ACM Trans. Math. Softw.*, 8(2), 210–218. <https://doi.org/10.1145/355993.356001>
- Laurie, D. P. (1997). Calculation of Gauss–Kronrod quadrature rules. *Math. Comput.*, 66(219), 1133–1145. <https://doi.org/10.1090/S0025-5718-97-00861-2>
- Revels, J., Lubin, M., & Papamarkou, T. (2016). Forward-mode automatic differentiation in Julia. *arXiv:1607.07892 [Cs.MS]*. <https://arxiv.org/abs/1607.07892>
- Xiao, H., & Gimbutas, Z. (2010). A numerical algorithm for the construction of efficient quadrature rules in two and higher dimensions. *Comput. Math. Appl.*, 59(2), 663–676. <https://doi.org/10.1016/j.camwa.2009.10.027>